



The Top Ten Things (we didn't cover)



I don't know about you, but
I think it could do with
more spam...



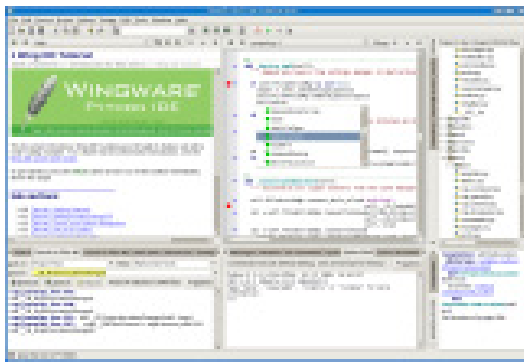
You've come a long way.

But learning about Python is an activity that never stops. The more Python you code, the more you'll need to learn new ways to do certain things. You'll need to master new tools and new techniques, too. There's just not enough room in this book to show you everything you might possibly need to know about Python. So, here's our list of the top ten things we didn't cover that you might want to learn more about next.

#1: Using a “professional” IDE

Throughout this book, you’ve used Python’s **IDLE**, which is great to use when first learning about Python and, although it’s a little quirky, can handle most programming tasks. It even comes with a built-in **debugger** (check out the Debug menu), which is surprisingly well equipped. Chances are, however, sooner or later, you’ll probably need a more full-featured integrated development environment.

One such tool worth looking into is the **WingWare Python IDE**. This professional-level development tool is specifically geared toward the Python programmer, is written by and maintained by Python programmers, and is *itself* written in Python. **WingWare Python IDE** comes in various licencing flavor: it’s free if you’re a student or working on an open source project, but you’ll need to pay for it if you are working within a for-profit development environment.



Written in Python by Python programmers for other Python programmers...what else could you ask for?

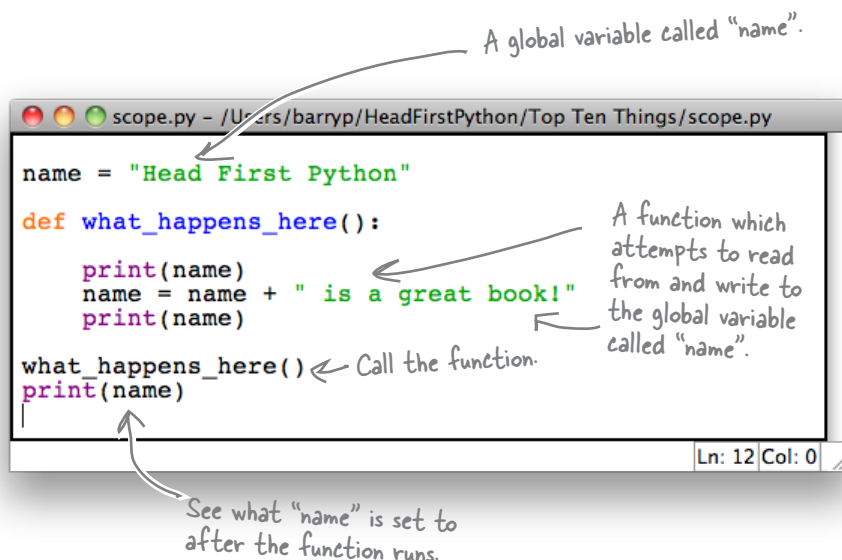
The WingWare Python IDE

More general tools also exist. If you are running Linux, the **KDevelop IDE** integrates well with Python.

And, of course, there are all those *programmer editors* which are often all you’ll ever need. Many Mac OS X programmers swear by the **TextMate** programmer’s editor. There’s more than a few Python programmers using **emacs** and **vi** (or its more common variant, **vim**). Your author is a huge fan of **vim**, but also spends large portions of his day using **IDLE** and the Python shell.

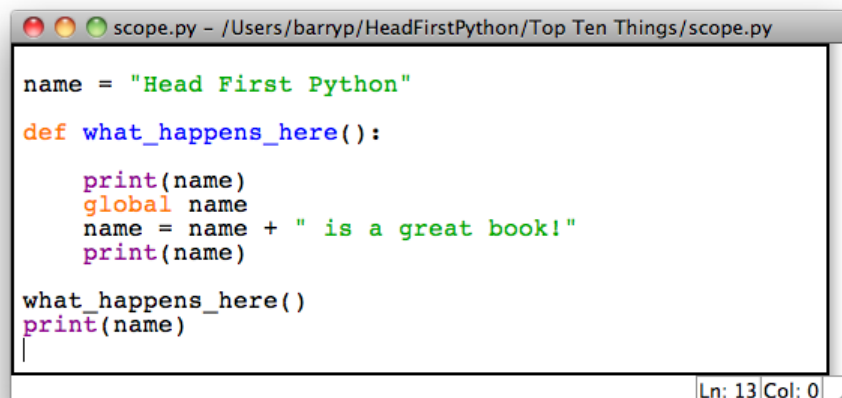
#2: Coping with scoping

Consider the following short program:



If you try to run this program, Python complains with this error message:
UnboundLocalError: local variable 'name' referenced before assignment... whatever that means!

When it comes to scope, Python is quite happy to let you access and read the value of a global variable within a function, **but you cannot change it**. When Python sees the assignment, it looks for a local variable called `name`, doesn't find it, and throws a *hissy fit* and an `UnboundLocalError` exception. To access *and* change a global variable, you must explicitly declare that's your intention, as follows:



Some programmers find this quite ugly. Others think it's what comes to pass when you watch Monty Python reruns while designing your programming language. No matter what everyone thinks: this is what we're stuck with! ☺

#3: Testing

Writing code is one thing, but **testing** it is quite another. The combination of the Python shell and IDLE is great for testing and experimenting with small snippets of code, but for anything substantial, a testing framework is a must.

Python comes with *two* testing frameworks out of the box.

The first is familiar to programmers coming from another modern language, because it's based on the popular *xUnit* testing framework. Python's `unittest` module (which is part of the standard library) lets you create test code, test data, and a test suite for your modules. These exist in separate files from your code and allow you to exercise your code in various ways. If you already use a similar framework with your current language, rest assured that Python's implementation is essentially the same.

The other testing framework, called `doctest`, is also part of the standard library. This framework allows you to take the output from a Python shell or IDLE session and use it as a test. All you need to do is copy the content from the shell and add it to your modules' *documentation strings*. If you add code like this to the end of your modules, they'll be ready for "doctesting":

```
if __name__ == "__main__":
    import doctest
    doctest.testmod() }
```

If your code is imported as a module, this code does *NOT* run. If you run your module from the command line, your tests run.

What do you mean: you can't hear me...I guess I should've tested this first, eh?

If you then run your module at your operating system's command line, your tests run. If all you want to do is import your module's code and not run your tests, the previous `if` statement supports doing just that.

For more on `unittest` and `doctest`, search the online Python documentation on the Web or via IDLE's *Help* menu.



#4: Advanced language features

With a book like this, we knew we'd never get to cover the entire Python language unless we tripled the page count.

And let's face it, no one would thank us for that!

There's a lot more to Python, and as your confidence grows, you can take the time to check out these advanced language features:

Anonymous functions: the `lambda` expression lets you create small, one-line, non-named functions that can be incredibly useful once you understand what's going on.

Generators: like iterators, generators let you process sequences of data. Unlike iterators, generators, through the use of the `yield` expression, let you minimize the amount of RAM your program consumes while providing iterator-like functionality on large datasets.

Custom exceptions: create your own exception object based on those provided as standard by Python.

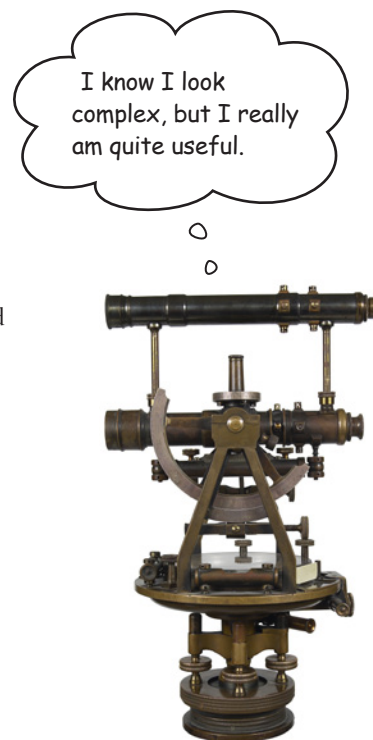
Function decorators: adjust the behavior of a preexisting function by hooking into its start-up and teardown mechanisms.

Metaclasses: custom classes that themselves can create custom classes. These are really only for the truly brave, although you did use a metaclass when you created your Sightings form using the *Django form validation framework* in Chapter 10.

Most (but not all) of these language features are primarily of interest to the Python programmer building tools or language extensions for use by other Python programmers.

You might never need to use some of these language features in your code, but they are all worth knowing about. Take the time to understand *when* and *where* to use them.

See **#10** of this appendix for a list of my favorite Python books (other than this one), which are all great starting points for learning more about these language features.



#5: Regular expressions

When it comes to working with textual data, Python is a bit of a natural. The built-in `string` type comes with so many methods that most of the standard string operations such as finding and splitting are covered. However, what if you need to extract a specific part of a string or what if you need to search and replace within a string based on a specific specification? It is possible to use the built-in string methods to implement solutions to these types of problems, but—more times than most people would probably like to admit to—using a **regular expression** works better.

Consider this example, which requires you to extract the area code from the `phone_number` string and which uses the built-in string methods:

```
area_code.py - /Users/barryp/HeadFirstPython/Top Ten Things/area_code.py

phone_number = "Home: (555) 265-2901"
start = phone_number.find('(') ← Find the opening "("
start = start+1
end = start+3 } ← Calculate where the area code is in the string
area_code = phone_number[start:end] ← Extract the area code.
print('The area code is: ' + area_code)
```

Ln: 12 Col: 0



Jeff Friedl's regular expression "bible", which is well worth a look if you want to learn more. Look up the "re" module in Python's docs, too.

This code works fine, but it *breaks* when presented with the following value for `phone_number`:

```
phone_number = "Cell (mobile): (555)-918-8271"
```

Why does this phone number cause the program to fail? Try it and see what happens...

When you use a **regular expression**, you can specify *exactly* what it is you are looking for and *improve the robustness* of your code:

```
area_code_re.py - /Users/barryp/HeadFirstPython/Top Ten Things/area_code_re.py

import re
phone_number = "Home: (555) 265-2901"
results = re.search('\((\d{3})\)', phone_number)
area_code = results.group(1)
print('The area code is: ' + area_code)
```

Ln: 10 Col: 0

This looks a little strange, but this regular expression is looking for an opening "(" followed by three digits and then a closing ")". This specification is much more likely to find the area code and won't break as quickly as the other version of this program.

#6: More on web frameworks

When it comes to building web applications, CGI works, but it's a little old-fashioned. As you saw in Chapter 10, Google's App Engine technology supports CGI, but also WSGI and a number of web framework technologies. If you aren't deploying to the cloud and prefer to roll your own, you have plenty of choices. What follows is a representative sample. My advice: try a few on for size and see which one works best for you.

Search for the following terms in your favorite search engine: **Django**, **Zope**, **TurboGears**, **Web2py**, and **Pylons**.



#7: Object relational mappers and NoSQL

Working with SQL-based databases in Python is well supported, with the inclusion of SQLite in the standard library a huge boon. Of course, the assumption is you are familiar with SQL and happy to use SQL to work with your data.

But what if you aren't? What if you *detest* SQL?

An **object relational mapper** (ORM) is a software technology that lets you use an underlying SQL-based database *without* having to know anything about SQL. Rather than the procedural interface based on the Python database API, ORMs provide an object-oriented interface to your data, exposing it via *method calls* and *attribute lookups* as opposed to columns and rows.

Many programmers find ORMs a much more natural mechanism for working with stored datasets and the Python community creates and supports a number of them.

One of the most interesting is **SQLAlchemy**, which is popular and included in a number of the web framework technologies discussed in #6. Despite being hugely popular anyway, **SQLAlchemy** is also interesting because it supports *both* Python 2 and Python 3, which makes it a standout technology (for now).

If you find yourself becoming increasingly frustrated by SQL, check out an ORM. Of course, you have already experienced a similar technology: *Google App Engine's datastore* API is very similar in style to those APIs provided by the major Python ORMs.



There's NoSQL, too.

In addition to database technologies that let you avoid working with the underlying SQL-based database, a new breed of technologies have emerged that let you *drop* your SQL database in its entirety. Known collectively as **NoSQL**, these data tools provide an alternative non-SQL API to your data and do not use an SQL-based database management system at all. As these technologies are relatively new, there's been more activity around Python 2 than Python 3, but they are still worth checking out. **CouchDB** and **MongoDB** are the two most closely associated with robust Python implementations. If you like working with your data in a Python dictionary and wished your database technology let you store your data in much the same way, then you need to take a look at **NoSQL**: *it's a perfect fit*.



#8: Programming GUIs

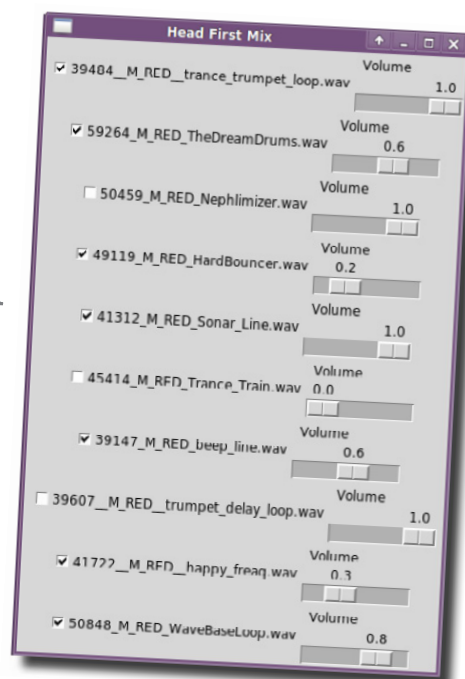
In this book, you've created text-based interfaces, web-based interfaces and interfaces that ran on Android devices. But what if you want to create a desktop application that runs on your or your user's desktop computer? Are you out of luck, or can Python help here, too?

Well...as luck would have it, Python comes preinstalled with a GUI-building toolkit called **tkinter** (shorthand for *Tk Interface*). It's possible to create a usable and useful graphical user interface (GUI) with **tkinter** and deploy it on Mac OS X, Windows, and Linux. With the latest version of **Tk**, your developed app takes on the characteristics of the underlying operating system, so when you run on Windows, your app looks like a Windows desktop app, when it run on Linux, it looks like a Linux desktop app, and so on.

You write your Python and **tkinter** code *once*, then run it anywhere and it just works. There are lots of great resources for learning to program with **tkinter**, with one of the best being the last few chapters of *Head First Programming*, but since plugging *that* book would be totally shameful, I won't mention it again.

Other GUI-building technologies do exist, with the **PyGTK**, **PyKDE**, **wxPython**, and **PyQT** toolkits coming up in conversation more than most. Be warned, however, that most of these toolkits target Python 2, although support for Python 3 is on its way. Search the Web for any of the project names to learn more.

Oh, look: it's one of the GUIs created in "Head First Programming"...and yes, I said I wouldn't mention THAT book again, but isn't this GUI beautiful? 😊



#9: Stuff to avoid

When it comes to stuff to avoid when using Python, there's a very short list. A recent tweet on *Twitter* went something like this:



Threads do indeed exist in Python but should be **avoided** where possible.

This has *nothing* to do with the quality of Python's threading library and *everything* to do with Python's implementation, especially the implementation known as **CPython** (which is more than likely the one you're running now). Python is implemented using a technology known as the **Global Interpreter Lock (GIL)**, which enforces a restriction that Python can only ever run on a single interpreter process, even in the presence of multiple processors.

What all this means to you is that your beautifully designed and implemented program that uses threads will never run faster on multiple processors even if they exist, because *it can't use them*. Your threaded application will run **serially** and, in many cases, run *considerably slower* than if you had developed the same functionality without resorting to threads.

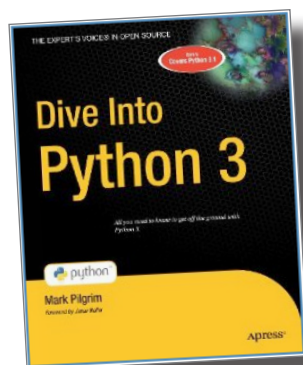
Main message: **don't use threads** with Python until the GIL restriction is removed...*if it ever is*.



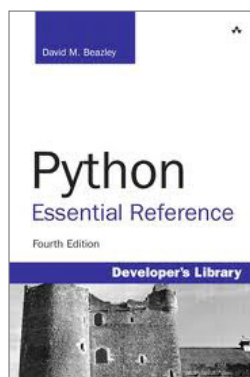
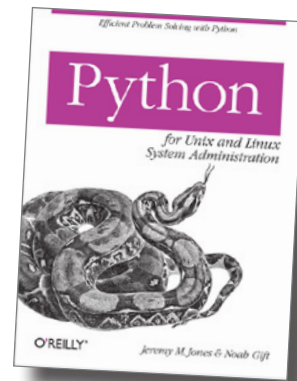
#10: Other books

There are lots of great books that cover Python in general, as well as specifically within a particular problem domain. Here is a collection of my favorite Python books, which we have no hesitation in recommending to you.

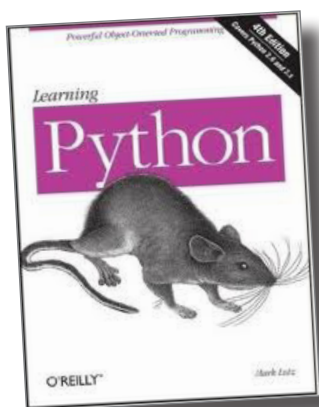
If you are a sysadmin, then this is the Python book for you.



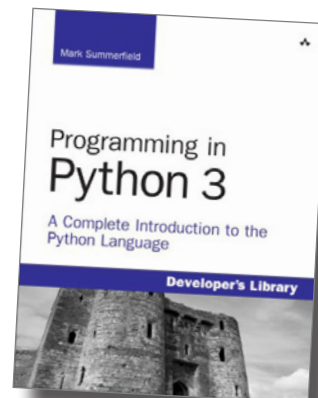
Includes a great case study involving the porting of a complex Python 2 module to Python 3.



The best desktop reference on the market



At 1,200 pages, this is the definitive language reference for Python: it's got everything in it!



Includes some big examples with big technology: XML, parsing and advanced language features.

Index

Symbols and Numbers

404 Error, from web server 242
405 Error, from web server 378
>>> (chevron, triple) IDLE prompt 4
: (colon)
 in for loop 16
 in function definition 29
 in if statement 20
, (comma) separating list items 7
{ } (curly braces)
 creating dictionaries 180
 creating sets 166
= (equal sign)
 assignment operator 7
 in function argument definition 63
(...) (parentheses)
 enclosing function arguments 29
 enclosing immutable sets 91
+ (plus sign) addition or concatenation operator 138
(pound sign) preceding one-line comments 38
@property decorator 250, 253, 285
? (question mark) parameter placeholder 321, 350
“...” or ‘...’ (quotes) enclosing each list item 7
“”” ...””” or “”...” (quotes, triple) enclosing comments 37
; (semicolon) separating statements on one line 38
[...] (square brackets)
 accessing dictionary items 180, 212
 accessing specific list items 9, 18
 enclosing all list items 7, 18

A

“a” access mode 110
access modes for files 110, 133

addition operator (+) 138
Alt-N keystroke, IDLE 5
Alt-P keystroke, IDLE 5
AndFTP app 288–289
Android apps
 accepting input from 278–282, 295, 304–307
 converting from Python code 424–430
 creating 274–277, 281–282
 data for. *See* JSON data interchange format
 integrating with SQLite 342–348
 running on phone 288–289
 scripting layer for. *See* SL4A
 troubleshooting 277
Android emulator
 installing and configuring 260–262
 running scripts on 264–265, 272–273, 283
Android Market 288
Android Virtual Device. *See* AVD
anonymous functions 439
append() method, lists 10, 14
apps. *See* Android apps; webapps
app.yaml file 356, 395
arguments for functions
 adding 52, 66–68
 optional 63–64, 134
arrays
 associative. *See* dictionaries
 similarity to lists 9–10, 17
as keyword 119, 138
assignment operator (=) 7
associative arrays. *See* dictionaries
attributes, class 190, 194, 212
authorization, user 389–393
AVD (Android Virtual Device) 261, 291

B

“batteries included” 32

BIFs. *See* built-in functions

BigTable technology 354, 359

blue text in IDLE 4

books

Dive Into Python 3 (CreateSpace) 445

Head First HTML with CSS & XHTML (O’Reilly) 374

Head First Programming (O’Reilly) 443

Head First SQL (O’Reilly) 313

Learning Python (O’Reilly) 445

Mastering Regular Expressions (O’Reilly) 440

Programming in Python 3 (Addison-Wesley Professional) 445

Python Essential Reference (Addison-Wesley Professional) 445

Python for Unix and Linux System Administration (O’Reilly) 445

braces. *See* curly braces

brackets, regular. *See* parentheses

brackets, square. *See* square brackets

BSD, running CGI scripts on 239

build folder 42

built-in functions (BIFs). *See also* specific functions

displayed as purple text in IDLE 4

help on 21

importing of, not needed 55

namespace for 55

number of, in Python 21

__builtins__ namespace 55, 71

C

cascading style sheet (CSS) 374–375

case sensitivity of identifiers 17

cgi-bin folder 234, 235

CGI (Common Gateway Interface) scripts 217, 235, 243, 253. *See also* WSGI

location of 234, 235

running 239

running from Android 264–265, 272–273, 283

sending data to 300–303

tracking module for 248–249

troubleshooting 242, 247–250

writing 236–238, 244–246

writing for Android. *See* SL4A

cgi library 300

cgibt module 248–249, 253

chaining

functions 146, 172

methods 142, 172

chevron, triple (>>>) IDLE prompt 4

chmod command 239, 253

classes 189–191

attributes of 190, 194, 212

benefits of 189

converting data to dictionary 285–286

defining 190–193, 194, 195–196

inherited 204–209, 212

instances of 190, 191, 194, 195–196

metaclasses 439

methods of 190, 195–196, 198–200

in modules 209, 212

class keyword 191, 212

close() method, database connection 315, 350

close() method, files 75, 103

code editors 35, 436. *See also* IDLE

colon (:)

- in for loop 16
- in function definition 29
- in if statement 20

comma (,) separating list items 7

comments 37–38

commit() method, database connection 315, 350

Common Gateway Interface scripts. *See* CGI scripts

comprehension, list 154–159, 172, 409–411, 432

concatenation operator (+) 138

conditional list comprehension 409–411, 432

conditions. *See* if/else statement

connection, database

closing 314, 315

creating 314, 315

`connect()` method, `sqlite3` 315, 350
 context management protocol 120
 Controller, in MVC pattern 221
 for GAE webapps 359, 370–373
 for webapps 234–238, 244–246
 “copied” sorting 144, 145–146, 172
 CREATE TABLE statement, SQL 317, 319–320
 CSS (cascading style sheet) 374–375
 CSV format, converting to Python data types 401–405
 curly braces (`{}`)
 creating dictionaries 180
 creating sets 166
`cursor()` method, database connection 315, 350
 custom code 131
 custom exceptions 439

D

data
 for Android apps. *See* JSON data interchange format
 bundling with code. *See* classes
 duplicates in, removing 161–163, 166–167
 external. *See* database management system; files
 for GAE webapps. *See* datastore, for GAE
 nonuniform, cleaning 148–153
 race conditions with 309–310
 Robustness Principle for 384–387
 searching for closest match 416–417
 sending to web server 275, 291
 sorting 144–147, 172
 storing. *See* persistence
 transforming, list comprehensions for 154–159
 database API 314–315, 350
 database management system 312
 closing connection to 314, 315
 commit changes to data 314, 315, 350
 connecting to 314, 315
 cursor for, manipulating data with 314, 315
 designing 316–318
 inserting data into 321, 324, 348
 integrating with Android apps 342–348
 integrating with webapps 327–341
 managing and viewing data in 326
 process for interacting with 314–315
 querying 322, 332–333
 rollback changes to data 314, 315, 350
 schema for 317
 SQLite for. *See* SQLite
 tables in 317, 319–320, 350
 data folder 234
 data interchange format. *See* JSON data interchange format
 data objects. *See also* specific data objects
 getting next item from 54
 ID for 54
 length of, determining 32
 names of. *See* identifiers
 datastore, for GAE 359–360, 380–383, 384–387, 395
 data types
 converting CSV data into 401–405
 converting strings to integers 54
 in datastore 360
 immutable 91, 103, 116, 138
 for JSON 285
 for list items 8, 12
 date and time data
 format compatibility issues 418–423
 property type for 362, 384–385
`db.Blob()` type 360
`db.DateProperty()` type 360
`db.IntegerProperty()` type 360
`db.StringProperty()` type 360, 385
`db.TimeProperty()` type 360
`db.UserProperty()` type 360, 390
 decision statement. *See* if/else statement
 decorators, function 439
`def` keyword 29, 191, 212
`dialogCreateAlert()` method, Android API 274, 276, 280
`dialogGetInput()` method, Android API 295, 304–306
`dialogGetResponse()` method, Android API 274, 276, 278, 280
`dialogGetSelectedItems()` method, Android API 278, 280
`dialogSetItems()` method, Android API 279, 280
`dialogSetNegativeButtonText()` method, Android API 274, 276
`dialogSetPositiveButtonText()` method, Android API 274, 276, 280

dialogSetSingleChoiceItems() method, Android API 274, 276

dialogShow() method, Android API 276, 280

dict() factory function 180, 212

dictionaries 178–182, 212

- accessing items in 180, 212
- compared to lists 179
- converting class data to 285–286
- creating 180, 182, 186
- dictionaries within 407–409
- keys for 178, 180, 212
- populating 180, 212
- reading CSV data into 403–404
- values of 178, 180, 212

dir() command 225

directory structure. *See* folder structure

dist folder 42

distribution

- creating 40–42
- updating 60–61, 65
- uploading to PyPI 48

Dive Into Python 3 (CreateSpace) 445

djangoforms.ModelForm class 368

Django Project

- Form Validation Framework 368–369, 395
- templates 363–366, 395

doctest framework 438

documentation for Python 3 3, 80, 103

dot notation 10, 194, 196

double quotes. *See* quotes

dump() function, pickle 133–134, 138

dumps() function, json 269, 272, 281, 291

dynamic content 216, 217

E

Eclipse editor 35

editors 35, 436. *See also* IDLE

elif keyword 108. *See also* if/else statement

else keyword. *See* if/else statement

emacs editor 35, 436

enable() function, cgibt 248, 253

end_form() function, yate 231, 233

entities, in datastore 360, 395

enumerate() built-in function 54

environ dictionary 300, 350

equal sign (=)

- assignment operator 7
- in function argument definition 63

errors. *See* exception handling; troubleshooting

exception handling 88–95, 103. *See also* troubleshooting

- benefits of 95, 100
- closing files after 114–115, 120–123
- custom exceptions 439
- defining with try/except statement 89, 91–94
- ignoring found errors 93–94
- IndexError exception 17
- IOError exception 103, 112–114, 117–119
- for missing files 96–98
- NameError exception 44, 118
- PickleError exception 133–134
- specific errors, checking for 101–102
- specific errors, details about 117–119
- TypeError exception 56–57, 116, 247–249, 283–285
- ValueError exception 78–79, 81–82, 103

exception objects 119, 138

except keyword. *See* try/except statement

execute() method, cursor 315, 322, 324, 350

extend() method, lists 10

F

F5 key, IDLE 39, 44, 49, 71

factory functions 166

favicon.ico file, for webapp 234

fetchall() method, cursor 322

fetchmany() method, cursor 322

fetchone() method, cursor 322

FieldStorage() method, cgi 244, 253, 296, 300, 350

files. *See also* persistence

- access modes for 110, 133
- appending data to 110
- checking for existence of 118

- closing 75, 110
- closing after exception 114–115, 120–123
- CSV format, converting to Python data types 401–405
- exceptions involving, determining type of 117–119
- flushing 110
- missing, exception handling for 96–98
- opening 75, 109–110
- opening in binary access mode 133
- reading data from 75–78, 142–143
- rewinding 76
- splitting lines in 77–78
- writing 110–113
- writing, custom formats for 126–130
- writing, default format for 124–125
- writing, pickle library for. *See* pickle library
- finally keyword 115, 138
- find() method, strings 84–86, 103
- Firefox, SQLite Manager for 326
- folder structure
 - for distribution 42
 - for GAE 356, 370
 - for webapps 234
- for loop 15–17, 32
 - compared to list comprehension 432
 - nesting 19–22
- forms, HTML 295
 - creating from template 296–299
 - Form Validation Framework for 368–369
 - input restrictions for 376–377, 384–387
 - sending data to CGI scripts 300–303
 - stylesheets for 374–375
- Form Validation Framework 368–369, 395
- 405 Error, from web server 378
- 404 Error, from web server 242
- Friedl, Jeff (author, *Mastering Regular Expressions*) 440
- from statement 46, 49
- functional programming concepts 157
- function decorators 439
- functions
 - adding arguments to 52, 66–68
 - anonymous 439
 - built-in. *See* built-in functions (BIFs)

- chaining 146, 172
- creating 28–30, 170–171
- optional arguments for 63–64, 134
- recursive 31
- sharing. *See* modules

G

- GAE (Google App Engine) 354
 - configuration and setup for 356–357
 - controller code for 370–373
 - data modeling with 360–362
 - datastore for 359, 380–383, 384–387, 395
 - deploying webapps to Google cloud 391
 - folder structure for 356, 370
 - form generation for 368–369
 - form input restrictions for 376–377, 384–387
 - form stylesheets for 374–375
 - MVC pattern used by 359
 - SDK for, downloading 355
 - troubleshooting 378
 - user authorization for 389–393
 - view for, designing 363–369
- GAE Launcher 357, 391, 395
- garbage collection 116
- generators 439
- get() method, GAE 370, 379, 395
- GET web request 370
- GIL (Global Interpreter Lock) 444
- glob module 237, 253
- Google App Engine. *See* GAE
- Google BigTable technology 354, 359
- GQL (Google Query Language) API 359
- green text in IDLE 4
- GUI (graphical user interface), building 443

H

- hashes. *See* dictionaries
- header() function, yate 231, 233
- Head First HTML with CSS & XHTML (O'Reilly) 374
- Head First Programming (O'Reilly) 443

Head First SQL (O'Reilly) 313

help() built-in function 80, 103

HTML

generating for webapp interface 230–231

learning 226

templates for, with Django 363–366

HTML forms. *See* forms, HTML

HTTP server 235

http.server module 235, 253

I

id() built-in function 54

IDE 436. *See also* IDLE

identifiers 7, 17, 32

IDLE 3–5, 32

colored syntax used in 4

indenting enforced in 4

preferences, setting 5

prompt in (>>>) 4

recalling and editing code statements 5

running or loading code in 39, 44, 49, 71

TAB completion 5

if/else statement 20, 32

elif keyword 108

in list comprehension 432

negating conditions in 86, 103

images folder 234

immutable data types 138

lists 91, 103, 116

numbers 116

strings 116

import statement 43, 46, 49, 71

include_footer() function, yate 230, 232, 233

include_header() function, yate 230, 232

indentation rules

enforced in IDLE 4

for for loops 16

for function definitions 29

for if statement 20

IndexError exception 17

index.html file, for webapp 234

inherited classes 204–209, 212

__init__() method 191, 212

in operator 16, 118, 138

“in-place” sorting 144, 145, 172

input

from Android apps 278–282, 295, 304–307

HTML forms for. *See* forms, HTML

from keyboard after screen prompt 413–414, 432

input() built-in function 413–414, 432

insert() method, lists 10, 14

INSERT statement, SQL 321, 324, 348

instances of classes 190, 191, 194, 195–196

int() built-in function 54

integers, converting strings to 54

interface. *See* View, in MVC pattern

IOError exception 103, 112–114, 117–119

I/O (input/output), handling. *See* files

isinstance() built-in function 20–22, 32

iterations

for loop 15–17, 19–22, 32

generating with range() function 54–56

while loop 16–17

J

JSON data interchange format 266–267, 291

API for, using 269–272

browser differences with 272

data types supported by 285

incompatibility with pickle data objects 284–285

K

KDevelop IDE 436

keys, in dictionary 178, 180, 212

keywords, displayed as orange text in IDLE 4

L

- lambda expression 439
- Learning Python (O'Reilly) 445
- len() built-in function 10, 32
- lib folder 42
- Linux
 - code editors for 35
 - GAE log messages on 378
 - IDEs for 436
 - installing Python 3 on 3
 - running CGI scripts on 239, 272
 - running GAE Launcher on 357
 - transferring files to Android device 288
- list() built-in function 54
- list comprehension 154–159, 172, 409–411, 432
- lists 32. *See also* data objects
 - adding items to 10–14
 - bounds checking for 17
 - classes inherited from 204–208
 - compared to dictionaries 179
 - compared to sets 167
 - creating 6–7, 54
 - data types in 8, 12
 - duplicates in, removing 161–163
 - extracting specific item from 175–176
 - getting next item from 54
 - identifiers for 7
 - immutable 91, 103, 116
 - iterating 15–17, 157
 - length of, determining 10, 32
 - methods for 10
 - nested, checking for 20–22
 - nested, creating 18–19
 - nested, handling 23–25, 28–31
 - numbered, creating 54
 - reading CSV data into 403–404
 - removing items from 10
 - similarity to arrays 9–10, 17
 - slice of 160, 172
- load() function, pickle 133, 138
- loads() function, json 269, 276, 280, 291

locals() built-in function 118, 138

loops. *See* iterations

M

Mac OS X

- code editors for 35
- GAE log messages on 378
- IDEs for 436
- installing Python 3 on 3
- running CGI scripts on 239, 272
- running GAE Launcher on 357
- transferring files to Android device 288
- __main__ namespace 45
- MANIFEST file 42
- mappings. *See* dictionaries
- Mastering Regular Expressions (O'Reilly) 440
- metaclasses 439
- methods 190. *See also* specific methods
 - chaining 142, 172
 - for classes 195–196, 198–200
 - creating 212
 - results of, as attributes 250, 253
 - self argument of 212
- ModelForm class, djangoforms 368
- Model, in MVC pattern 221
 - for GAE webapps 359, 360–362
 - for webapps 222–225
- Model-View-Controller pattern. *See* MVC pattern
- modules 34–36, 71
 - adding functionality to 50–52
 - classes in 209, 212
 - creating 35
 - distribution for, creating 40–42
 - distribution for, updating 60–61, 65
 - distribution for, uploading to PyPI 48
 - importing 43–44, 46
 - loading in IDLE 39, 49, 71
 - locations for 38, 49
 - namespaces for 45–46, 71
 - in Python Standard Library 36
 - third-party 36

Monty Python 17
 multiple inheritance 209
 MVC (Model-View-Controller) pattern 221, 232, 253, 359
 Controller 234–238, 244–246, 370–373
 Model 222–225, 360–362
 View 226–233, 363–369

N

NameError exception 44, 118
 names. *See* identifiers
 namespaces 45–46, 71
 next() built-in function 54
 NoSQL 359, 442
 NotePad editor 35
 not in operator 161–162
 not keyword 86, 103
 numbered lists 54

O

object relational mapper. *See* ORM (object relational mapper)
 objects. *See* data objects
 open() built-in function 75, 103, 109–110
 orange text in IDLE 4
 ORM (object relational mapper) 442
 os module 76, 300, 350

P

para() function, yate 231, 233
 parentheses (...)
 enclosing function arguments 29
 enclosing immutable lists 91
 pass statement 93, 103
 persistence 105
 pickle library for 132–137
 reading data from files 222–224
 writing data to files 110–113, 222–224
 PickleError exception 133–134

pickle library 132–137, 138
 data modeling using 222–224
 incompatibility with JSON data types 284–285
 transferring data to a database 321–325
 plus sign (+) addition or concatenation operator 138
 pop() method, lists 10, 175–176
 post() method, GAE 379–383, 395
 POST web request 379
 pound sign (#) preceding one-line comments 38
 print() built-in function 10, 32, 124–125
 disabling automatic new-line for 56, 71
 displaying TAB character with 56
 writing to a file 110, 128, 138
 Programming in Python 3 (Addison-Wesley Professional) 445
 properties, in datastore 360, 395
 @property decorator 250, 253, 285
 purple text in IDLE 4
 put() method, GAE 395
 .pyc file extension 42, 49
 .py file extension 35
 PyPI (Python Package Index) 36
 registering on website 47
 uploading distributions to 48
 uploading modules to 209
 Python 2
 compared to Python 3 17
 raw_input() built-in function 432
 running on Android smartphones 258–259, 291
 using with Google App Engine 355
 Python 3
 compared to Python 2 17
 documentation for 3, 80, 103
 editors for 35, 436
 installing 3
 interpreter for. *See* IDLE
 learning 445
 python3 command
 building a distribution 41
 checking for Python version 3
 installing a distribution 41
 uploading a new distribution 68

Python Essential Reference (Addison-Wesley Professional) 445

Python for Unix and Linux System Administration (O'Reilly) 445

Python, Monty 17

Python Package Index. *See* PyPI

Python Standard Library 36

Q

querying a database 322, 332–333

question mark (?) parameter placeholder 321, 350

quotes (“...” or ‘...’) enclosing each list item 7

quotes, triple (“...” or ‘...’) enclosing comments 37

R

“r” access mode 110

race conditions 309–310

radio_button() function, yate 231, 233

range() built-in function 54–56, 71

raw_input() built-in function 432

readline() method, files 76, 103, 142

recursion 31

regular brackets. *See* parentheses

regular expressions 440

re module 440

remove() method, lists 10

render() function, template 364, 366

Robustness Principle 384–387

rollback() method, database connection 315, 350

runtime errors 88. *See also* exception handling; troubleshooting

S

schema, database 317

scoping of variables 437

Scripting Layer for Android. *See* SL4A

scripts. *See* CGI scripts; SL4A

sdist command 41

seek() method, files 76, 103

SELECT/OPTION, HTML tag 376

SELECT statement, SQL 322, 332–333

self argument 192–193, 212

self.request object 379, 395

self.response object 372, 379, 395

semicolon (;) separating statements on one line 38

set() built-in function 166, 172

sets 166, 167, 172

setup() built-in function 40

setup.py file 40, 71

single quotes. *See* quotes

SL4A (Scripting Layer for Android) 258, 291

adding Python to 263

Android apps, creating 274–277

automatic rotation mode, setting 264

documentation for 274

installing 262

Python versions supported 258–259

slice of a list 160, 172

smartphones, apps on. *See* Android apps

sorted() built-in function 144–147, 153, 158, 172

sort() method, lists 144–145, 153, 172

split() method, strings 77–78, 80–81, 103, 142

SQLAlchemy 442

SQLite 313, 350

closing connection to 314, 315

committing data to 314, 315

connecting to 314, 315

cursor for, manipulating data with 314, 315

designing database 316–318

inserting data into 321, 324, 348

integrating with Android apps 342–348

integrating with webapps 327–341

managing data in 326

process for interacting with 314–315

querying 322, 332–333

rollback changes to data 314

schema for database 317

tables in, creating 319–320

- sqlite3 command 326
- sqlite3 library 313, 315, 350
- SQLite Manager, for Firefox 326
- SQL (Structured Query Language) 313, 350. *See also* NoSQL; SQLite; ORM
- square brackets ([...])
 - accessing dictionary items 180, 212
 - accessing specific list items 9, 18
 - enclosing all list items 7, 18
- standard error (sys.stderr) 248, 291
- standard input (sys.stdin) 291
- Standard Library, Python 36
- standard output (sys.stdout) 126–128, 291
- start_form() function, yate 231, 233
- start_response() function, yate 230, 232
- static content 216, 217
- static folder 370
- str() built-in function 119, 138
- strings
 - concatenating 138
 - converting other objects to 119
 - converting to integers 54
 - displayed as green text in IDLE 4
 - finding substrings in 84–86
 - immutable 116
 - sorting 148
 - splitting 77–78, 80–81
 - substitution templates for 230, 253
- strip() method, strings 108, 138, 142
- Structured Query Language. *See* SQL
- stylesheets for HTML forms 374–375
- suite 16, 29, 32
- sys module 291
- sys.stdout file 126–128, 138

T

- TAB character, printing 56
- TAB completion, IDLE 5
- tables, database 317, 319–320, 350
- target identifiers, from split() method 77, 91
- .tar.gz file extension 42
- Template class 230, 253
- template module 364
- templates folder 234, 370
- templates for GAE 363–366, 395
- testing code 438
- TextMate editor 35, 436
- third-party modules 36
- threads 444
- time data
 - format compatibility issues 418–423
 - property type for 362, 384–385
- time module 419, 432
- Tk Interface (tkinter) 443
- traceback 88, 103. *See also* exception handling; troubleshooting
- tracing code 58–59
- triple chevron (>>>) IDLE prompt 4
- triple quotes (“”” ... ””” or ‘’ ‘... ’’) enclosing comments 37
- troubleshooting. *See also* exception handling
 - 404 Error, from web server 242
 - 405 Error, from web server 378
 - Android apps 277
 - GAE webapps 378
 - testing code 438
 - tracing code 58–59
- try/except statement 89, 93–94, 101–102, 103
 - finally keyword for 115, 138
 - with statement and 120–123
- tuples 91, 103, 116
- TypeError exception 56–57, 116, 247–249, 283–285

U

u_list() function, yate 231, 233
 unittest module 438
 urlencode() function, urllib 291
 urllib2 module 291
 urllib module 291
 urlopen() function, urllib2 291
 user authorization 389–393
 user input. *See* forms, HTML; input
 UserProperty() type, db 390

V

ValueError exception 78–79, 81–82, 103
 values, part of dictionary 178, 180, 212
 variables, scope of 437
 vi editor 35, 436
 View, in MVC pattern 221
 for GAE webapps 359, 363–369
 for webapps 226–233
 vim editor 436

W

“w” access mode 110
 “w+” access mode 110
 “wb” access mode 133

webapps 215–217, 253
 controlling code for 221, 234–238, 244–246
 data modeling for 221, 222–225
 designing with MVC 221
 design requirements for 218–220
 directory structure for 234
 Google App Engine for. *See* GAE
 input data, sending to CGI scripts 300–303
 input forms for. *See* forms, HTML
 SQLite used with 327–341
 view for 221, 226–233
 Web-based applications. *See* webapps
 web frameworks 441. *See also* CGI; WSGI
 web request 216, 253, 395
 web response 216–217, 253, 395
 web server 216–217, 235
 Web Server Gateway Interface (WSGI) 356, 370. *See also* CGI scripts
 while loop 16–17, 55
 WingIDE editor 35
 WingWare Python IDE 436
 with statement 120–123, 138
 WSGI (Web Server Gateway Interface) 356, 370. *See also* CGI scripts

Y

yate (Yet Another Template Engine) library 226–233
 yield expression 439

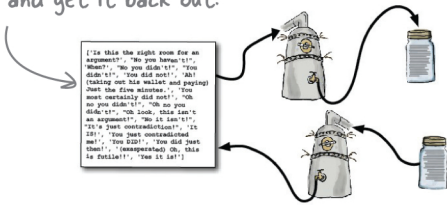
Head First Python

Python/Programming Languages

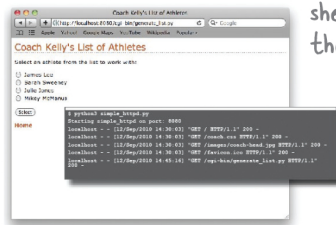
What will you learn from this book?

Ever wished you could learn Python from a book? *Head First Python* helps you learn the language through a unique method that goes beyond syntax and how-to manuals. You'll quickly grasp Python's fundamentals, then move on to persistence, exception handling, web development, SQLite, data wrangling, and Google App Engine. You'll also learn how to write mobile apps for Android, all thanks to the power that Python gives you. *Head First Python* is a complete learning experience that will help you become a bona fide Python programmer.

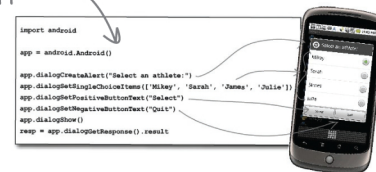
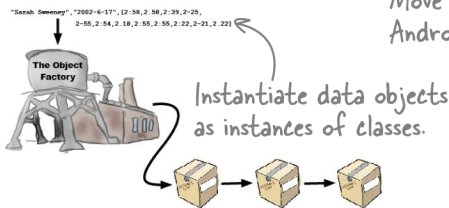
Get your data into a pickle...
and get it back out.



Port your scripts from the Python
shell to the Web.



Move your webapp to phones as an
Android app.



Why does this book look so different?

We think your time is too valuable to waste struggling with new concepts. Using the latest research in cognitive science and learning theory to craft a multi-sensory learning experience, *Head First Python* uses a visually rich format designed for the way your brain works, not a text-heavy approach that puts you to sleep.

“Head First Python is a great introduction to not just the Python language, but Python as it’s used in the real world. The book goes beyond the syntax to teach you how to create applications for Android phones, Google’s App Engine, and more.”

— David Griffiths,
author and Agile coach

“Where other books start with theory and progress to examples, Head First Python jumps right in with code and explains the theory as you read along. The breadth of examples and explanation cover the majority of what you’ll use in your job every day.”

— Jeremy Jones, coauthor of
Python for Unix and Linux
System Administrators

US \$49.99

CAN \$57.99

ISBN: 978-1-449-38267-4



Safari
Books Online

Free online edition
for 45 days with
purchase of this book.
Details on last page.

O'REILLY
oreilly.com
headfirstlabs.com